



UFRN - UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
DIMAp - DEPARTAMENTO DE MATEMÁTICA E INFORMÁTICA APLICADA

Unidade 1: Lista de exercícios.

Componente Curricular:

DIM0404 - Cálculo Numérico para Ciência da Computação

Docente:

Rafael Beserra Gomes

Discentes:

Iara Kelly Lima da Silva

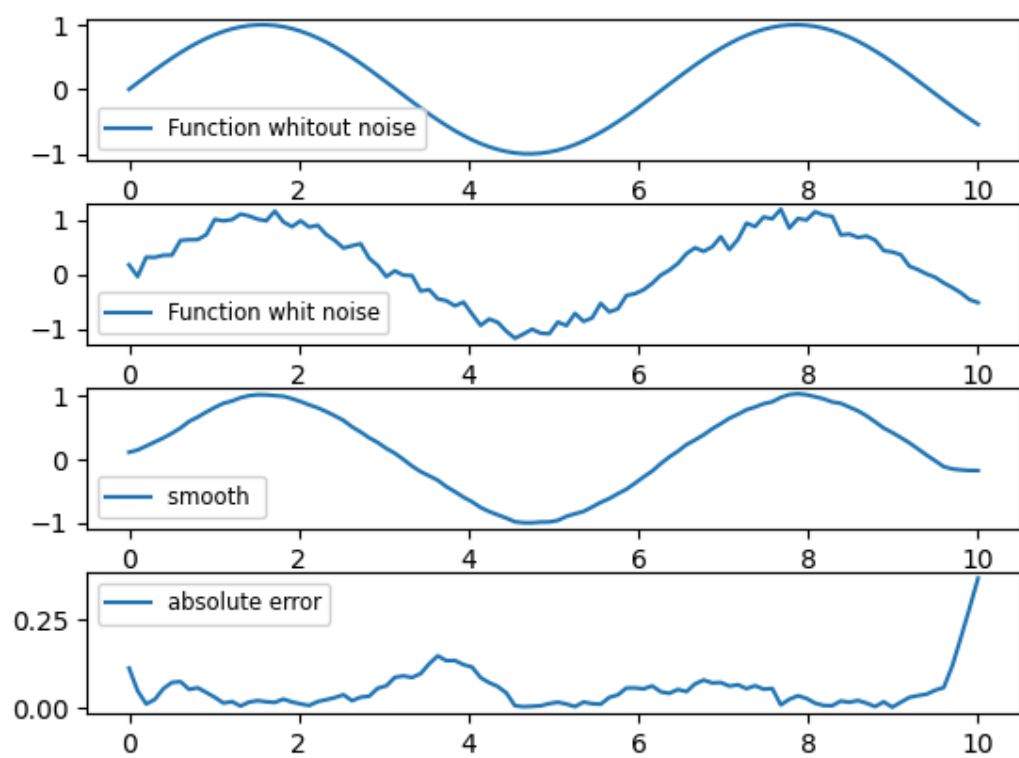
NATAL-RN

2026

Introdução

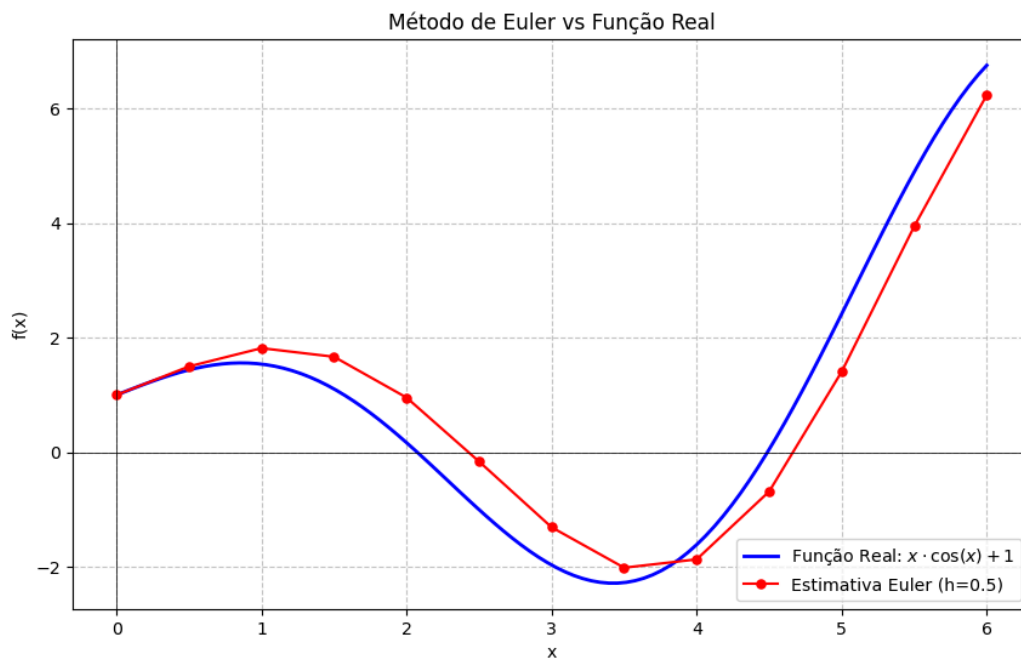
Questão 2:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(start=0, stop=10, num=100)
5 y = np.sin(x)
6
7 plt.subplots_adjust(hspace=0.3)
8 plt.subplot(4,1, 1)
9 plt.plot(x, y, label='Function whitout noise') #plotagem da função
10 plt.legend(fontsize='small') #legenda
11 plt.savefig('figure.png')
12
13 noise = np.random.normal(loc=0, scale=0.1, size=len(x)) #geração de ruído
14 #esta pegando cada ponto da curva anterior e gerando um intervalo normal, com media 0 e desvio padrão 1
15 y_noise = y + noise #adição do ruído à função
16
17 plt.subplot(4, 1,2)
18 plt.plot(x, y_noise, label='Function whit noise')
19 plt.legend(fontsize='small') #legenda
20 plt.savefig('figure.png')
21
22 y_conv= np.convolve(y_noise, np.ones(10)/10,
23 mode='same') #convolução da função com ruído
24
25 plt.subplot(4,1,3)
26 plt.plot(x, y_conv, label='smooth ')
27 plt.legend(fontsize='small') #legenda
28 plt.savefig('figure.png')
29
30 err_abs = np.abs(y - y_conv) #erro absoluto
31 plt.subplot(4,1,4)
32 plt.plot(x, err_abs, label='absolute error')
33 plt.legend(fontsize='small') #legenda
34 plt.savefig('figure.png')
35
36 plt.subplots_adjust(hspace=1.5)
```



Questão 4:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 #função original
5 def f_real(x):
6     return x * np.cos(x) + 1
7
8 #derivada conhecida (f'(x))
9 def df(x):
10     return np.cos(x) - x * np.sin(x)
11
12 x0, y0 = 0, 1          # Ponto inicial conhecido f(0) = 1
13 x_final = 6            # Fim do intervalo
14 h = 0.5                # Tamanho do passo (pode diminuir para aumentar a precisão)
15
16 # Listas para armazenar os pontos estimados
17 x_euler = [x0]
18 y_euler = [y0]
19
20 # Loop do Método de Euler
21 x_atual = x0
22 y_atual = y0
23
24 while x_atual < x_final:
25     # Calcula a inclinação no ponto atual
26     inclinacao = df(x_atual)
27
28     # Próximo y = y atual + passo * derivada
29     y_proximo = y_atual + h * inclinacao
30     x_proximo = x_atual + h
31
32     # Armazena os resultados
33     x_euler.append(x_proximo)
34     y_euler.append(y_proximo)
35
36     # Atualiza para a próxima iteração
37     x_atual = x_proximo
38     y_atual = y_proximo
39
40 # Preparação dos dados para o gráfico da função real (curva suave)
41 x_curva = np.linspace(0, 6, 100)
42 y_curva = f_real(x_curva)
43
44 plt.figure(figsize=(10, 6))
45 plt.plot(x_curva, y_curva, label='Função Real:  $x \cdot \cos(x) + 1$ ', color='blue', linewidth=2)
46 plt.plot(x_euler, y_euler, 'ro-', label=f'Estimativa Euler (h={h})', markersize=5)
47
48 plt.title('Método de Euler vs Função Real')
49 plt.xlabel('x')
50 plt.ylabel('f(x)')
51 plt.legend()
52 plt.grid(True, linestyle='--', alpha=0.7)
53 plt.axhline(0, color='black', linewidth=0.5)
54 plt.axvline(0, color='black', linewidth=0.5)
55 plt.show()
```



Representação Numérica

Questão 5:

a) $6201_7 \rightarrow x_3$

$$6201_7 \rightarrow x_{10} = 6 \cdot 7^3 + 2 \cdot 7^2 + 0 \cdot 7^1 + 1 \cdot 7^0$$

$$6 \cdot 343 + 2 \cdot 49 + 0 + 1$$

$$2058 + 98 + 1 = 2157$$

$2157_{10} \rightarrow x_3 = 2221220_3$

Handwritten conversion of 2157 to base 3:

$$2157 \div 3 = 719 \text{ R } 0$$

$$719 \div 3 = 239 \text{ R } 2$$

$$239 \div 3 = 79 \text{ R } 2$$

$$79 \div 3 = 26 \text{ R } 1$$

$$26 \div 3 = 8 \text{ R } 2$$

$$8 \div 3 = 2 \text{ R } 2$$

$$2 \div 3 = 0 \text{ R } 2$$

Result: $2157_{10} = 2221220_3$

Handwritten conversion of 2157 to hexadecimal:

$$2157 \div 16 = 134 \text{ R } 13$$

$$134 \div 16 = 8 \text{ R } 6$$

$$8 \div 16 = 0 \text{ R } 8$$

Result: $2157_{10} = 86D_{16}$

b) $EBA_{36} \rightarrow x_2$

$$\begin{aligned} EBA_{16} \rightarrow x_{10} &= E \cdot 16^2 + B \cdot 16^1 + A \cdot 16^0 \\ &= 14 \cdot 256 + 11 \cdot 16 + 10 \\ &= 3584 + 176 + 10 = 3770 \end{aligned}$$

$$\begin{array}{r} 3770 \overline{) 12} \\ 2 \\ \hline 17 \\ 16 \\ \hline 1 \\ 10 \\ 10 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1885 \overline{) 12} \\ 181 \\ \hline 08 \\ 8 \\ \hline 05 \\ 4 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 942 \overline{) 12} \\ 942 \\ \hline 8 \\ 14 \\ 14 \\ \hline 02 \\ 2 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 471 \overline{) 12} \\ 471 \\ \hline 4 \\ 07 \\ 6 \\ \hline 11 \\ 11 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 235 \overline{) 12} \\ 235 \\ \hline 2 \\ 03 \\ 2 \\ \hline 15 \\ 14 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 117 \overline{) 12} \\ 117 \\ \hline 10 \\ 17 \\ 16 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 58 \overline{) 12} \\ 58 \\ \hline 4 \\ 29 \\ 28 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 29 \overline{) 12} \\ 29 \\ \hline 2 \\ 09 \\ 8 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 14 \overline{) 12} \\ 14 \\ \hline 7 \\ 6 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 3 \overline{) 12} \\ 3 \\ \hline 3 \\ 3 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \overline{) 12} \\ 1 \\ \hline 1 \\ 1 \\ \hline 0 \end{array}$$

$$\begin{aligned} EBA_{16} &\rightarrow x_2 \\ &\rightarrow 111010111010_2 \end{aligned}$$

c) $111000001100010111101_2 \rightarrow x_{16} \rightarrow 1C18BD_{16}$

						$2^0 = 1$
						$2^1 = 2$
						$2^2 = 4$
						$2^3 = 8$
						$2^4 = 16$

0001	1100	0001	1000	1011	1101	
x 8421	8421	8421	8421	8421	8421	
0001	8400	0001	8000	8021	8401	
1	12	1	8	11	13	
1	C	1	8	B	D	



Questão 6:

$$99 \Rightarrow x_2 \Rightarrow 1100011$$

$$\begin{array}{r}
 99 \cancel{12} \\
 - 8 \cancel{49} \cancel{12} \\
 \hline
 19 \quad 4 \quad 24 \quad \cancel{12} \\
 - 18 \quad 09 \quad 2 \quad 12 \quad \cancel{12} \\
 \hline
 1 \quad 8 \quad 04 \quad 12 \quad 6 \quad \cancel{12} \\
 \hline
 \quad 1 \quad 4 \quad 2 \quad 0 \quad 3 \quad \cancel{12} \\
 \hline
 \quad \quad 0 \quad 0 \quad 0 \quad 2 \quad 1. \\
 \hline
 \quad \quad \quad 1
 \end{array}$$

$$1100011$$

(1) Sinal magnitude

$$-99 = 11100011$$

(3) Complemento de 2

↳ Somar 1 ao comp. de 1

(2) Complemento de 1

$$\begin{array}{r}
 10011100 \\
 + 00000001 \\
 \hline
 10011101
 \end{array}$$

$$-99 = 10011100$$

Questão 7:

$$\text{Sinal} = 1$$

$$\text{int} = 382$$

$$\text{fracção} = 0,775$$

$$382 \begin{array}{l} \underline{2} \\ - 2 \\ \hline 0 \end{array} \quad \begin{array}{l} 191 \underline{2} \\ - 181 \\ \hline 10 \end{array} \quad \begin{array}{l} 95 \underline{2} \\ - 88 \\ \hline 7 \end{array} \quad \begin{array}{l} 47 \underline{2} \\ - 44 \\ \hline 3 \end{array} \quad \begin{array}{l} 23 \underline{2} \\ - 22 \\ \hline 1 \end{array} \quad \begin{array}{l} 11 \underline{2} \\ - 10 \\ \hline 1 \end{array} \quad \begin{array}{l} 5 \underline{2} \\ - 4 \\ \hline 1 \end{array} \quad \begin{array}{l} 2 \underline{2} \\ - 2 \\ \hline 0 \end{array}$$

$$382 = 101111110$$

$$\left. \begin{array}{l} 0,775 \cdot 2 = 1,55 (1) \\ 0,55 \cdot 2 = 1,10 (1) \\ 0,1 \cdot 2 = 0,2 (0) \\ 0,2 \cdot 2 = 0,4 (0) \\ 0,4 \cdot 2 = 0,8 (0) \\ 0,8 \cdot 2 = 1,6 (1) \\ 0,6 \cdot 2 = 1,2 (1) \end{array} \right\} \text{não repete}$$

$$382,775 = x_2 = 101111110.110001100110011...$$

normalizando:

$$1,01111110110001100110011... \times 2^8$$

→ mantissa

Exponente

$$8 + 127 = 135$$

$$135 \begin{array}{l} \underline{2} \\ - 128 \\ \hline 7 \end{array} \quad \begin{array}{l} 67 \underline{2} \\ - 64 \\ \hline 3 \end{array} \quad \begin{array}{l} 33 \underline{2} \\ - 32 \\ \hline 1 \end{array} \quad \begin{array}{l} 16 \underline{2} \\ - 16 \\ \hline 0 \end{array} \quad \begin{array}{l} 8 \underline{2} \\ - 8 \\ \hline 0 \end{array} \quad \begin{array}{l} 4 \underline{2} \\ - 4 \\ \hline 0 \end{array} \quad \begin{array}{l} 2 \underline{2} \\ - 2 \\ \hline 0 \end{array} \quad \begin{array}{l} 1 \end{array}$$

$$\text{exponente} = 10000111$$

$$-382,775 \Rightarrow x_2$$

$$= 11000011101111110110001100110011$$

Questão 8:

a)

Cada botão é atribuído a uma potência de 2

Δ	1	0000	0000	0000	0001
X	2	0000	0000	0000	0010
$\square$	4	0000	0000	0000	0100
O	8	0000	0000	0000	1000
:	:				

Exemplo Se o jogador apertar Δ e O teremos
 $1 + 8 = 9 \rightarrow (0000\ 0000\ 0000\ 1001)$

b)

Para economizar espaço não salvamos strings, como "SetaUP", "Triângulo", pois nesse caso temos 1 byte por carácter. Salvaremos em binário puro

Para recuperar fazemos um And do inteiro lido e o valor do botão.

Zeros de Funções

Questão 3:

```
--- Bisseção em [-4, -3] ---
Iteração 0: x = -3.500000, f(x) = 15.638936
Iteração 1: x = -3.250000, f(x) = -7.295967
Iteração 2: x = -3.375000, f(x) = 3.012535
Iteração 3: x = -3.312500, f(x) = -2.419881
Iteração 4: x = -3.343750, f(x) = 0.225353
Iteração 5: x = -3.328125, f(x) = -1.114827
Iteração 6: x = -3.335938, f(x) = -0.449150
Iteração 7: x = -3.339844, f(x) = -0.113004
Iteração 8: x = -3.341797, f(x) = 0.055898
Iteração 9: x = -3.340820, f(x) = -0.028623
Iteração 10: x = -3.341309, f(x) = 0.013620
Iteração 11: x = -3.341064, f(x) = -0.007506
Iteração 12: x = -3.341187, f(x) = 0.003056
Iteração 13: x = -3.341125, f(x) = -0.002225
Iteração 14: x = -3.341156, f(x) = 0.000416

--- Newton-Raphson (x0 = -1) ---
Iteração 0: x = -1.507931, f(x) = -0.396236
Iteração 1: x = -1.499233, f(x) = 0.000465

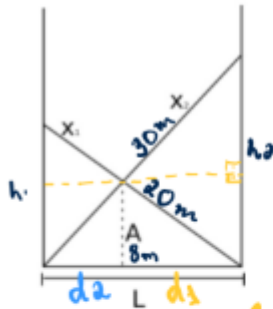
--- Secante (x0=2, x1=3) ---
Iteração 0: x = 2.502214, f(x) = -0.481212
Iteração 1: x = 2.492345, f(x) = 0.028467
Iteração 2: x = 2.492896, f(x) = -0.000008

--- Regula Falsi em [4, 5] ---
Iteração 0: x = 4.524027, f(x) = -17.984293
Iteração 1: x = 4.673487, f(x) = -4.039135
Iteração 2: x = 4.703926, f(x) = -0.772719
Iteração 3: x = 4.709637, f(x) = -0.143182
Iteração 4: x = 4.710691, f(x) = -0.026373
Iteração 5: x = 4.710885, f(x) = -0.004853
Iteração 6: x = 4.710921, f(x) = -0.000893

=====
Raízes encontradas:
-3.3412
-1.4992
2.4929
4.7109
```

Código disponível no arquivo zipado.

Questão 4:



$$\begin{aligned}x_1^2 &= L^2 + h_1^2 \\h_1 &= \sqrt{x_1^2 - L^2} \\h_1 &= \sqrt{20^2 - L^2}\end{aligned}$$

$$\begin{aligned}x_2^2 &= L^2 + h_2^2 \\h_2 &= \sqrt{x_2^2 - L^2} \\h_2 &= \sqrt{30^2 - L^2}\end{aligned}$$



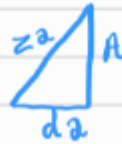
$$\frac{A}{h_1} = \frac{d_1}{L}$$

$$d_1 = \frac{A \cdot L}{h_1}$$



$$\frac{A}{h_2} = \frac{d_2}{L}$$

$$d_2 = \frac{A \cdot L}{h_2}$$



$$d_1 + d_2 = L$$

$$\frac{A \cdot L}{h_1} + \frac{A \cdot L}{h_2} = L \quad (\div L)$$

$$\frac{A}{h_1} + \frac{A}{h_2} = 1$$

$$A \cdot \left(\frac{1}{h_1} + \frac{1}{h_2} \right) = 1$$

$$\frac{1}{h_1} + \frac{1}{h_2} = \frac{1}{h}$$

$$\frac{1}{\sqrt{400-L^2}} + \frac{1}{\sqrt{900-L^2}} = \frac{1}{8}$$

$$f(L) = \frac{1}{\sqrt{400-L^2}} + \frac{1}{\sqrt{900-L^2}} - \frac{1}{8}$$

Resolução segue no repositório.

$$L \approx 16,21$$

```

1  import numpy as np
2
3
4  def f(x):
5      return 1/np.sqrt(400 - x**2) + 1/np.sqrt(900 - x**2) - 0.125
6
7  #2. newton method
8
9  def df(x):
10     return x / (400 - x**2)**(1.5) + x / (900 - x**2)**(1.5)
11
12  def newthon_method(x, precision):
13
14     while True:
15         if x >= 20:
16             x = 19.9
17
18         next_x = x - (f(x)/ (df(x))) # calc next x
19
20         #stopped if
21         if abs(f(x)) < precision:
22             print(f"The root is approximately: {next_x:.2f}")
23             break
24
25         x = next_x
26
27  newthon_method(15, 0.0001)

```

Sistemas Lineares

Questão 9:

Executando Gauss-Jordan

$$\left[\begin{array}{ccc|ccc} \textcircled{1} & 3 & 4 & 1 & 0 & 0 \\ -2 & -5 & -6 & 0 & 1 & 0 \\ 3 & 8 & 11 & 0 & 0 & 1 \end{array} \right] \Rightarrow \left[\begin{array}{ccc|ccc} \textcircled{1} & 3 & 4 & 1 & 0 & 0 \\ 0 & 1 & 2 & 2 & 1 & 0 \\ 0 & -1 & -1 & -3 & 0 & 1 \end{array} \right] \begin{array}{l} l_2 = l_2 - (-2)l_1 \\ l_3 = l_3 - 3l_1 \end{array}$$

$$\Rightarrow \begin{array}{l} l_3 = l_3 - (-l_2) \\ \left[\begin{array}{ccc|ccc} 1 & 3 & 4 & 1 & 0 & 0 \\ 0 & \textcircled{1} & 2 & 2 & 1 & 0 \\ 0 & 0 & 1 & -1 & 1 & 1 \end{array} \right] \end{array}$$

$$\Rightarrow \begin{array}{l} l_3 = l_3 - (-l_2) \\ \left[\begin{array}{ccc|ccc} 1 & 3 & 4 & 1 & 0 & 0 \\ 0 & \textcircled{1} & 2 & 2 & 1 & 0 \\ 0 & 0 & 1 & -1 & 1 & 1 \end{array} \right] \end{array}$$

$$\Rightarrow \begin{array}{l} l_2 = l_2 - 2l_3 \\ l_1 = l_1 - 4l_3 \\ \left[\begin{array}{ccc|ccc} 1 & 3 & 0 & 5 & -4 & -4 \\ 0 & 1 & 0 & 4 & -1 & -2 \\ 0 & 0 & \textcircled{1} & -1 & 1 & 1 \end{array} \right] \end{array}$$

$$\Rightarrow \begin{array}{l} l_1 = l_1 - 3l_2 \\ \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & -7 & -1 & 2 \\ 0 & \textcircled{1} & 0 & 4 & -1 & -2 \\ 0 & 0 & 1 & -1 & 1 & 1 \end{array} \right] \end{array}$$

$$\left[\begin{array}{ccc} 1 & 3 & 4 \\ -2 & -5 & -6 \\ 3 & 8 & 11 \end{array} \right]^{-1} = \left[\begin{array}{ccc} -7 & -1 & 2 \\ 4 & -1 & -2 \\ -1 & 1 & 1 \end{array} \right]$$

Questão 10:

```
1 import numpy as np
2 import sys
3
4 def gauss_jordan(M):
5     n = M.shape[0]
6     # Matriz aumentada [A|b]
7     for i in range(n):
8         # Pivotamento parcial: busca o maior elemento na coluna para reduzir erros
9         pivot_row = i + np.argmax(abs(M[i:, i]))
10        M[[i, pivot_row]] = M[[pivot_row, i]]
11
12        if M[i, i] == 0:
13            return None # Matriz singular
14
15        # Normaliza a linha do pivô para que o pivô seja 1
16        M[i] = M[i] / M[i, i]
17
18        # Elimina os outros elementos da coluna i (acima e abaixo do pivô)
19        for j in range(n):
20            if i != j:
21                M[j] = M[j] - M[j, i] * M[i]
22
23        # A solução está na última coluna
24        return M[:, -1]
25
26 def calcular_residuo(A_orig, b_orig, x):
27     # r = b - Ax
28     return b_orig - np.dot(A_orig, x)
29
30 # --- Fluxo Principal ---
31 if len(sys.argv) < 2:
32     print("Uso: python esc.py m1.in")
33     sys.exit(1)
34
35 with open(sys.argv[1]) as arq:
36     lines = arq.readlines()
37
38 # n é o primeiro valor do arquivo
39 n = int(lines[0].strip())
40 # Matriz aumentada completa do arquivo
41 dados = np.array([[float(x) for x in line.split()] for line in lines[1:]])
42
43 # Separamos A e b originais para o refinamento
44 A_orig = dados[:, :n]
45 b_orig = dados[:, n]
46
```

```

1
2 # 1. Primeira solução com Gauss-Jordan
3 M_copy = dados.copy()
4 x_inicial = gauss_jordan(M_copy)
5
6 # 2. Refinamento Iterativo
7 # Calculamos o resíduo:  $r = b - Ax$ 
8 residuo = calcular_residuo(A_orig, b_orig, x_inicial)
9
10 # Resolvemos o sistema  $A * d = r$  para achar a correção 'd'
11 # Criamos uma nova matriz aumentada  $[A \mid \text{residuo}]$ 
12 M_refinamento = np.hstack((A_orig.copy(), residuo.reshape(-1, 1)))
13 d = gauss_jordan(M_refinamento)
14
15 # Nova solução:  $x_{\text{novo}} = x_{\text{antigo}} + d$ 
16 x_final = x_inicial + d
17
18 # --- Análise de Impacto ---
19 norma_inicial = np.linalg.norm(residuo)
20 residuo_final = calcular_residuo(A_orig, b_orig, x_final)
21 norma_final = np.linalg.norm(residuo_final)
22
23 print(f"Solução inicial: {x_inicial}")
24 print(f"Solução após refinamento: {x_final}")
25 print("-" * 30)
26 print(f"Norma do Resíduo Inicial: {norma_inicial:.2e}")
27 print(f"Norma do Resíduo Final: {norma_final:.2e}")
28 print(f"Melhoria: {((norma_inicial - norma_final) / norma_inicial * 100):.4f}%")

```

Teste com primeiro arquivo:

```

kelly@pedro-VivoBook-ASUSLaptop-X513EQ-X513EQ:~/calc_num/escalonador$ python3 esc.py m1.in
Solução inicial: [ 1.22816879  1.25013022 -0.81479816 -0.72067807]
Solução após refinamento: [ 1.22816879  1.25013022 -0.81479816 -0.72067807]
-----
Norma do Resíduo Inicial: 1.87e-15
Norma do Resíduo Final: 9.02e-16
Melhoria: 51.7752%
kelly@pedro-VivoBook-ASUSLaptop-X513EQ-X513EQ:~/calc_num/escalonador$

```

Teste com segundo arquivo:

```

kelly@pedro-VivoBook-ASUSLaptop-X513EQ-X513EQ:~/calc_num/escalonador$ python3 esc.py m2.in
 1.9532642  1.72165137  0.65723047 -2.48400545  1.23856427  0.20696197
-2.79496151 -0.03093238 -2.3746026  1.78673324  0.61980398 -1.55578219
 0.32767843 -1.29861098  0.28665368 -2.69225092  1.78177839  0.17665243
 2.07728561  2.29589925  1.68777407 -2.06887221 -0.30219051 -1.21955583
-2.95611562  0.33138727 -0.38577993 -1.37014957 -0.03698123 -0.7790926
-0.42363989  2.05749891  2.20067197 -0.16757111 -2.63675342  0.97480985
-0.69994292  1.59977831 -0.48745126  0.69434139  0.17715055 -1.14066936
-2.21713612 -0.54880038 -0.70396256 -0.27368783]
-----
Norma do Resíduo Inicial: 3.80e-13
Norma do Resíduo Final: 9.74e-14
Melhoria: 74.3452%

```